

Modeling and Control of Cyber-Physical Systems: Project Activity II Report

Coronelli Marco s348664
Givogre Riccardo s349452
Guidetti Matteo s348900

1 Introduction

This report explores the modeling and distributed control of a cyber-physical system (CPS) composed of multiple magnetic levitation (maglev) units operating in coordination. Specifically, the system consists of seven agents: one designated as the leader and six as followers. Each unit is represented by a linearized state-space model obtained around a chosen equilibrium point. These agents form a multi-agent system tasked with achieving cooperative tracking, where the overall objective is to drive the global disagreement error to zero through decentralized control.

The project involves designing and comparing two types of distributed control protocols: one using neighborhood-based observers and another relying on local observers. These control strategies are evaluated under different reference signal scenarios, including step, ramp, and sinusoidal trajectories generated by the leader. The study also investigates how various communication network structures and control parameters, such as the coupling gain and the weighting matrices, affect the overall system performance.

The first step is designing the leader node such that it will act in the system as a reference generator, giving the follower nodes a trajectory to track. The leader node must be an autonomous system. The dynamics of the leader node are defined as:

$$\dot{x}_0 = Ax_0, \quad y_0 = Cx_0$$

making it in fact an autonomous system, that guides other 6 identical maglev systems, described each by these state-space equations:

$$\dot{x}_i = Ax_i + Bu_i, \quad y_i = Cx_i$$

where:

$$A = \begin{bmatrix} 0 & 1 \\ 880.87 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ -9.9453 \end{bmatrix}, \quad C = [708.27 \quad 0], \quad D = [0]$$

Since the state variables are not directly measurable ($C \neq I$) our study focuses on 2 different approaches:

- **A Distributed Regulator based on Local Observers**, where each of the 6 non-leader maglev systems uses local information and outputs from other agents .
- **A Distributed Regulator based on a Distributed Neighborhood Observer structure**, where the agents exchange information with their neighbors creating a robust and connected network.

We evaluate the performance of the two proposed control structures by applying classical reference profiles: step, ramp, and sinusoidal; which must be implicitly embedded in the dynamics of the leader node, since it is an autonomous system and cannot receive external inputs.

To achieve this, the leader is designed as a closed-loop system with a state-feedback law:

$$u_0 = -K_0x_0$$

which results in the modified autonomous dynamics:

$$\dot{x}_0 = (A - BK_0)x_0$$

By appropriately choosing the eigenvalues of the matrix $(A - BK_0)$, the leader node can internally generate different reference trajectories:

- **Step:** $\lambda = \{0, -1\}$
- **Ramp:** $\lambda = \{0, 0\}$
- **Sinusoid:** $\lambda = \{+j, -j\}$

The gain matrix K_0 is computed using either pole placement (place) or Ackermann's formula (acker), depending on the desired dynamic behavior. In particular, the acker function is used when assigning equal eigenvalues (e.g., in the ramp case with $\lambda = \{0, 0\}$), as it handles this situation more reliably than place.

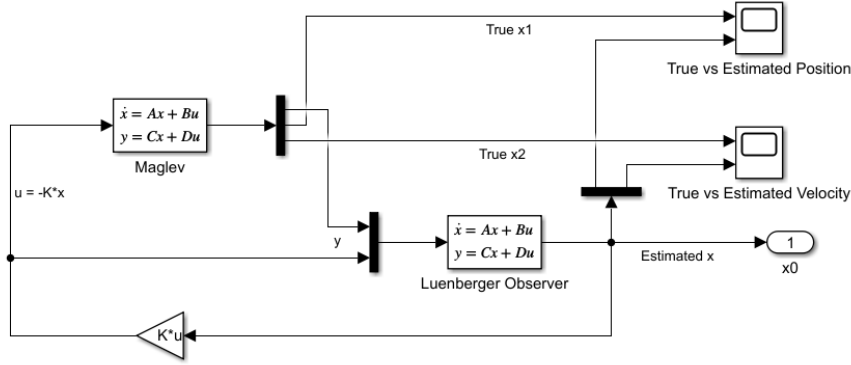


Figure 1: Structure of the leader node

The second state-space module is the observer, which is necessary because we cannot directly measure the internal states of the system. Therefore, we make use of a Luenberger observer, which provides us with an estimate of the system's state based on the available input and output data.

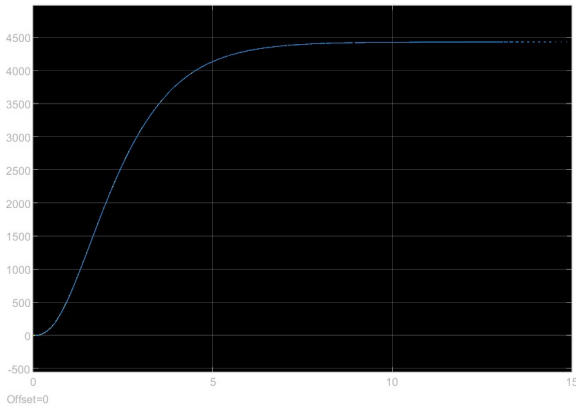
Although the estimate may not be exact, the observer uses the system model along with the difference between the actual and estimated outputs to continually refine its prediction. This enables accurate reconstruction of the state vector, allowing effective state feedback control even when only partial system information is available through measurements.

Moreover, in our simulation environment, we have access to the true states, which allows us to directly compare them with the estimated ones. To do this, we concatenate our matrix C with the identity matrix I_2 , resulting in the augmented output matrix $\begin{bmatrix} C \\ I_2 \end{bmatrix}$. With this augmentation, our output becomes the tuple $[y, x_1, x_2]$, where the last two values represent the true states. These are used to compare against the follower agents' estimated states to assess the accuracy of our state estimation.

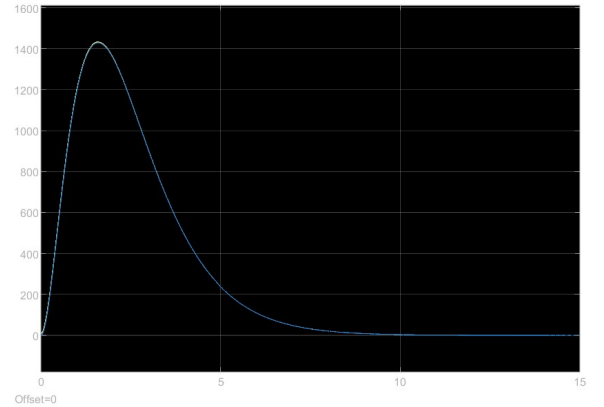
The observer gain is not shown in the graph, since it was implemented mathematically modifying the matrices of the state space block as follows:

$$A = A - LC, \quad B = [L \quad B], \quad C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

The observer gain L was computed in MatLab using the command `L = place(A', C', eigs)';`.



True vs Estimated Position (x_1)



True vs Estimated Velocity (x_2)

Figure 2: Leader node True vs Estimated states

As we can see there's little difference between the estimated and true state. We expected it to be larger, as the observer provides only an approximation, but this will probably be amplified in the follower node, that relies on estimated information from others. This should be evident in the chain topology and a little attenuated in the fully connected one, because the follower nodes are more connected to each other so approximation should swiftly converge to zero.

In the next sections we will discuss two methods to implement follower agents that track the trajectory given by the leader node.

2 Distributed Regulator based on Local Observers

This approach relies on placing an observer within each individual agent, enabling the estimation of internal states based on local measurements. By assuming that all outputs of the single agents are available to their respective observers, each observer can reconstruct the agent's state in real time, despite the impossibility of direct state measurement.

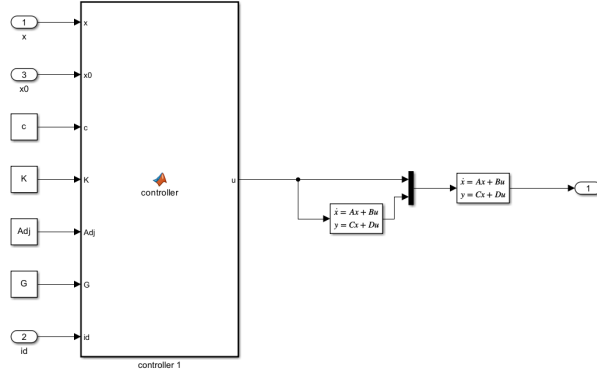


Figure 3: Single agent structure

Inside each controller is the control protocol of each node, expressed as:

$$u_i = cK\xi_i$$

where:

- ε_i is the neighborhood tracking error specific of a node, estimated as:

$$\xi_i = \sum_{j=1}^n a_{ij}(x_j - x_i) + g_i(x_0 - x_i)$$

- c is the coupling gain, that needs to be over a certain value if we want our error to converge to zero:

$$c \geq \frac{1}{2\min_{i \in \mathcal{N}} \text{Re}(\lambda_i)}$$

(in our case c is chosen equal to the minimum needed)

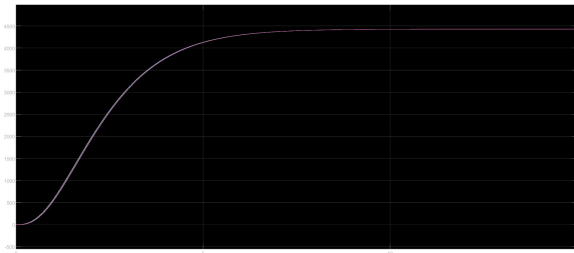
- K is the control gain defined as $K = R^{-1}B^TP$, here P is the unique positive definite solution to the algebraic Riccati equation (or ARE):

$$A^TP + PA + Q - PBR^{-1}B^TP = 0$$

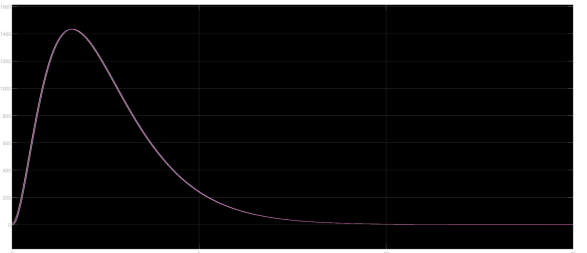
and the matrices Q and R are design matrices selected by the user.

The other variables introduced as inputs in the controller are needed to infer the structure of the network, their importance will be clear when we will discuss network structures in the next sections.

Here the observers are once again simplified cutting the feedback loop but implementing it mathematically modifying the matrix $A - BK_0$, B , C , D into $A - BK_0 - cFC$, $[C \ cF]$, I_2 , $\emptyset_{2,2}$, where F is a matrix found by pole placing (not to be confused for the F we will see in the next section).



True vs Estimated Positions (x_1)



True vs Estimated Velocities (x_2)

Figure 4: True states vs Estimated followers' states

In this picture we confront the true states of the system with the states estimated for each one of the agents, it's important to note our images are generated based on a mesh topology for reasons we discuss later in our report.

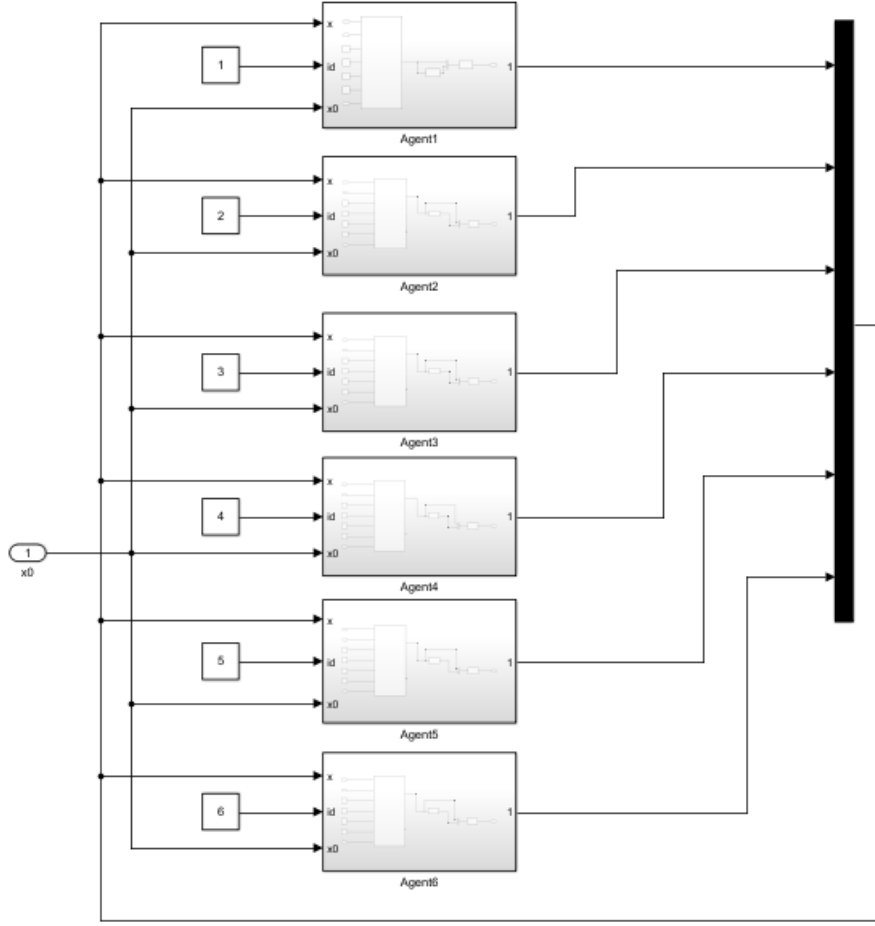


Figure 5: Local Observer Agent Structure

3 Distributed Regulator based on a distributed Neighborhood Observer structure

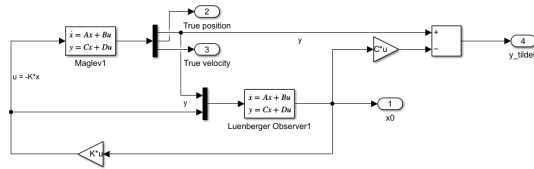


Figure 6: Leader node structure

In this approach we maintain the leader node structure (although we add another exit to it, that estimates the $\tilde{y}$ of the node, as shown in figure 27) and we also don't touch the single agent controller. What differentiates the two approaches is the way in which they implement the observer: in this case, instead of a local observer reserved for only one node, we provide a cooperative distributed observer that requires the estimates of the neighboring nodes to calculate its value $\hat{x}$:

$$\dot{\hat{x}} = A\hat{x}_i + Bu_i - cF\xi_i$$

where $F \in \mathbb{R}^{n \times p}$ is the observer gain, designed as $F = PC^T R^{-1}$ where P this time is the solution to the ARE:

$$AP + PA^T + Q - PC^T R^{-1} CP = 0$$

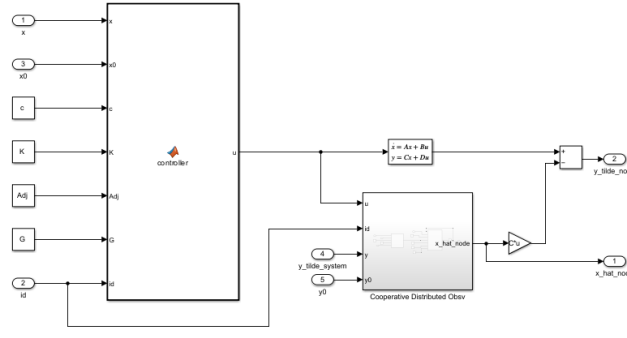


Figure 7: Single agent in the Neighbor Observer Agent Structure

This distinction is important because now the new values will depend on the estimation of the state variables not on the state variables themselves.

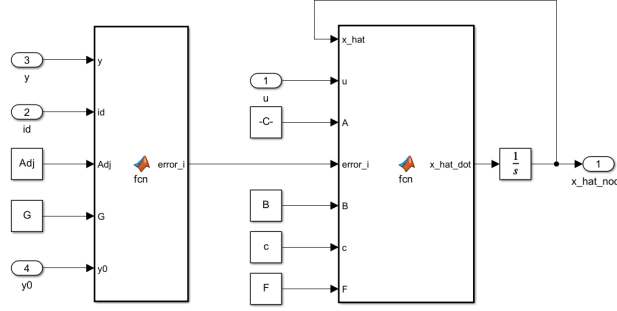


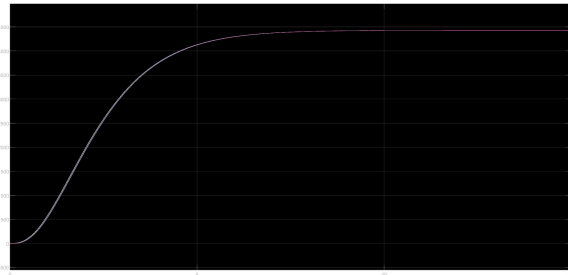
Figure 8: Cooperative Distributed Observer

In this figure we show what's inside the Cooperative Distributed Observer, we find a block that estimates the value of ξ_i and passes it to the block that inputs it in the equation:

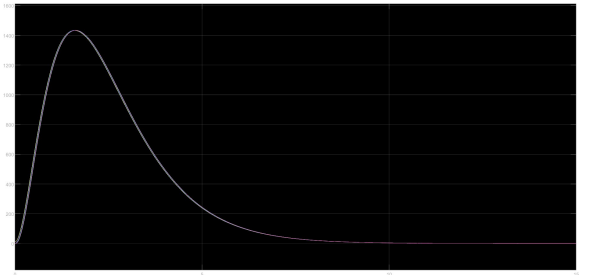
$$\dot{\hat{x}}_i = A\hat{x}_i + Bu_i + cF\xi_i$$

This is the the local closed loop dynamics of each node.

As we observed with the local observer structure, here we compare the true states of the system with the estimated states produced by our distributed cooperative neighbor observer.



True vs Estimated Positions (x_1)



True vs Estimated Velocities (x_2)

Figure 9: True states vs Estimated followers' states

This also highlights the advantages of the mesh topology: while some nodes are more isolated and may update their estimates more quickly, others have fewer neighbors and update more slowly. Interestingly, even the more loosely connected nodes benefit, as the neighbor measurements they rely on tend to be statistically closer to the true system states due to the averaging effect across the network.

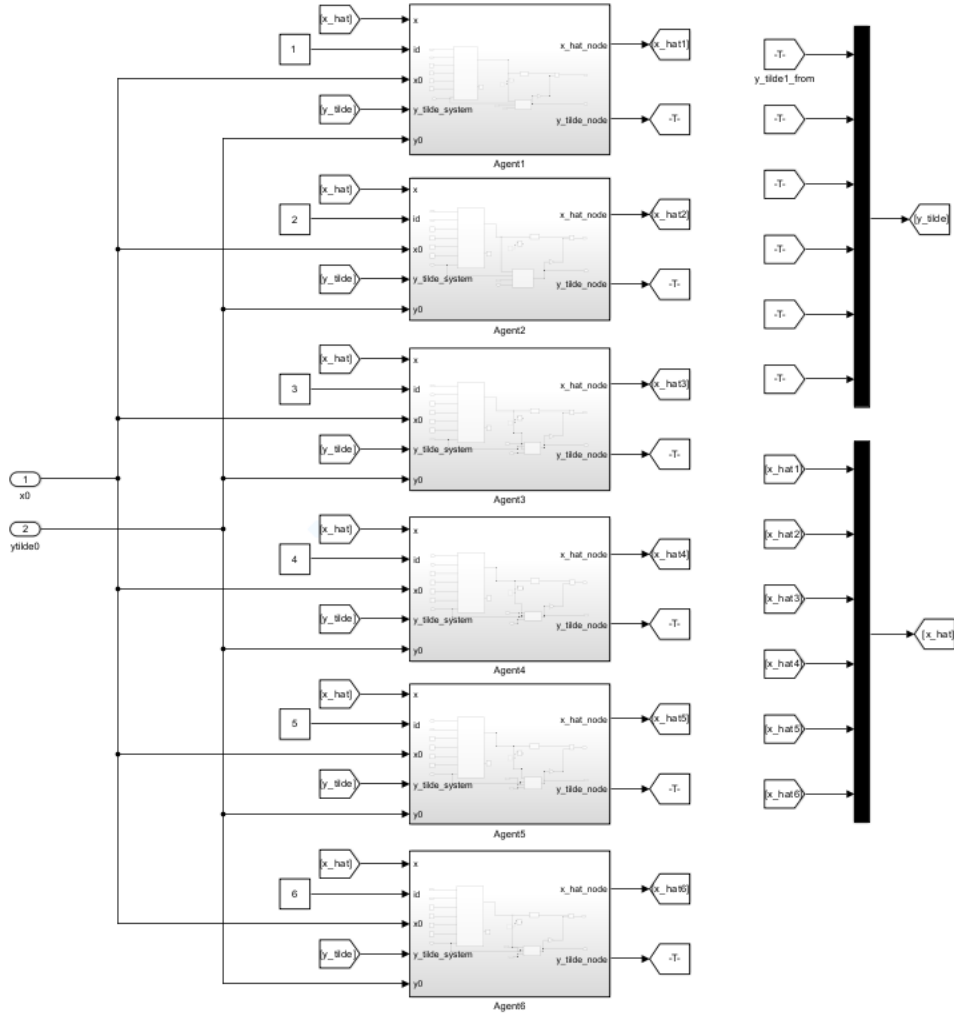
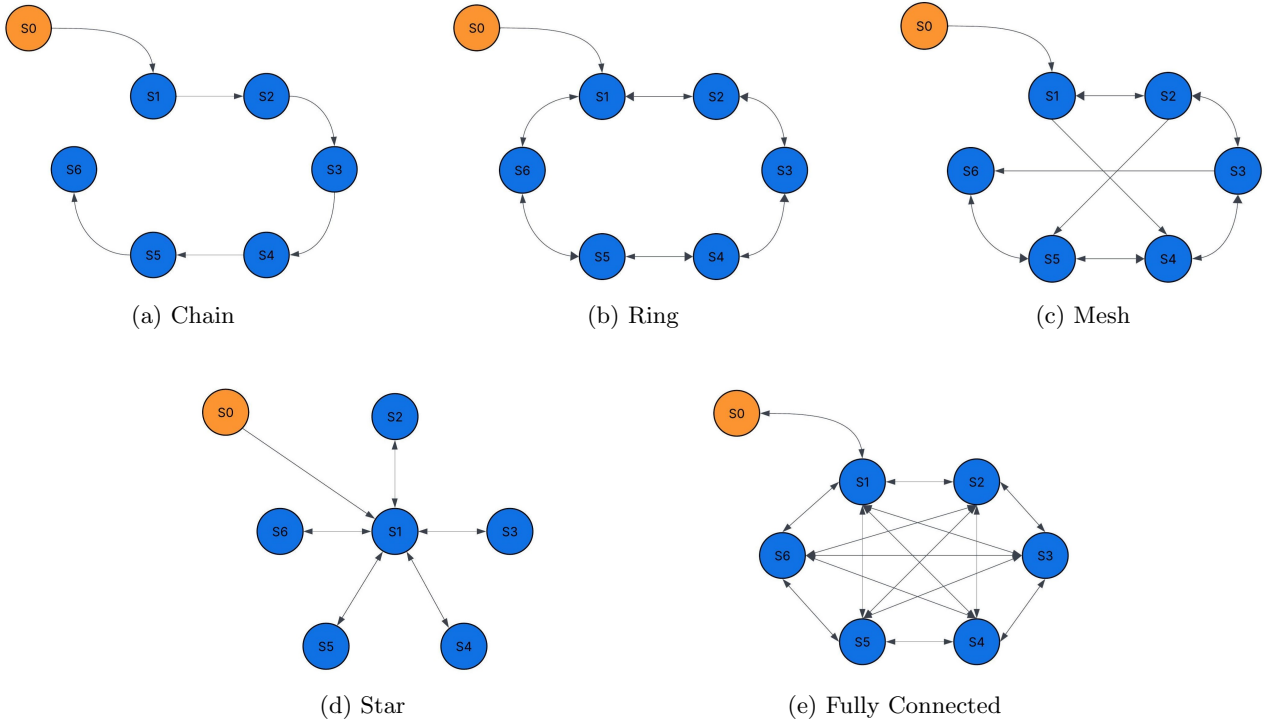


Figure 10: Neighbor Observer Agent Structure

4 Topologies

For our next analysis we try to implement different topologies so as to study the difference between each of them and find strengths and potential points of failure.



In a chain topology, nodes are lined up one after another, forming a straight path. It's simple but fragile, if one node in the middle fails, the whole line breaks. Communication slows down the further apart two nodes are. In our particular case this is seen as the first and sixth node are separated and do not precisely converge as seen in the pictures.

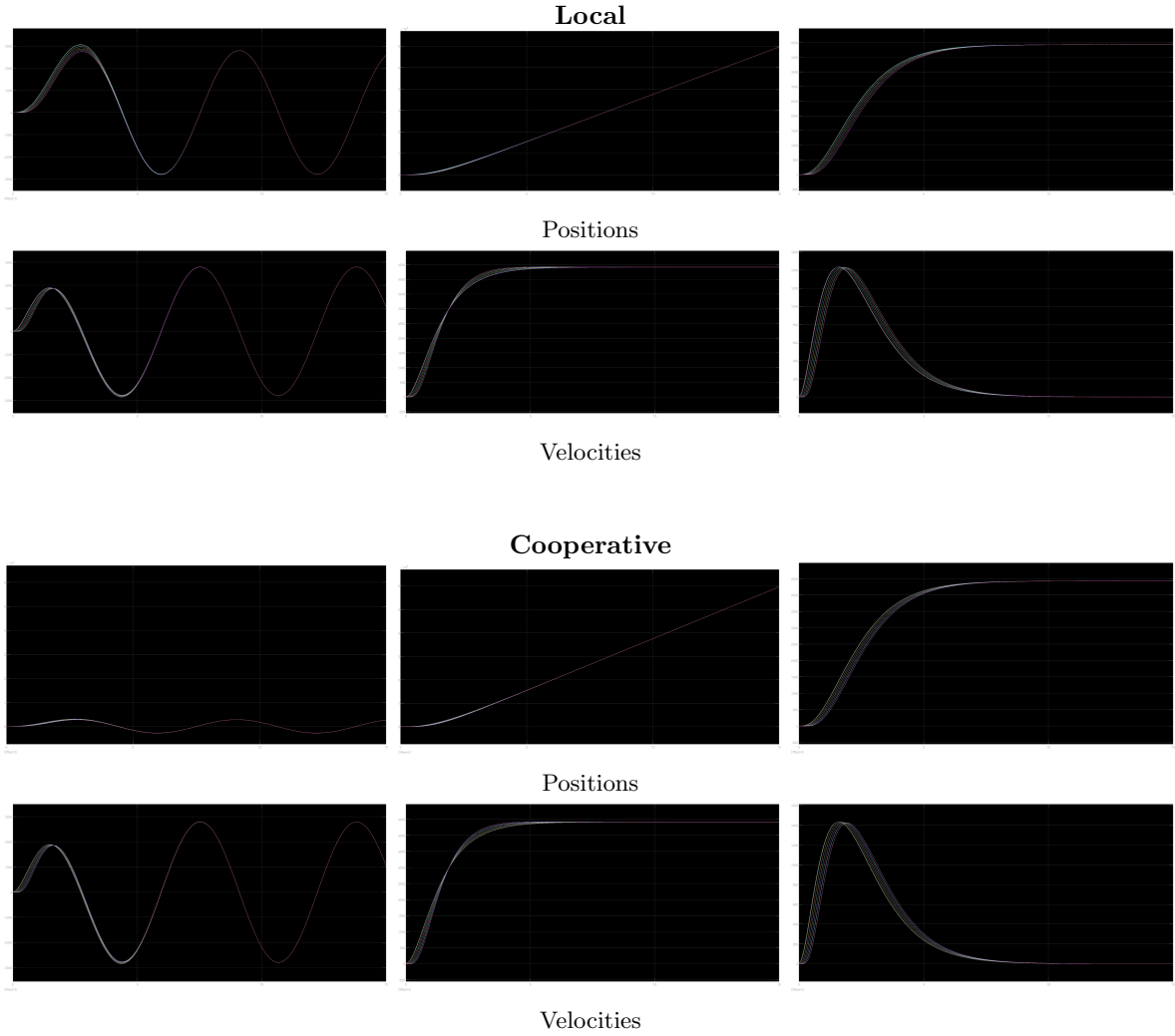


Figure 12: Chain topology under local and cooperative estimation strategies.

We chose to show only the graphs of the chain topology because it most clearly highlights the differences in trajectory between agents. In particular, we can clearly observe the delay of each node in response to the three elementary signals (step, ramp, and sine) for both position and velocity tracking.

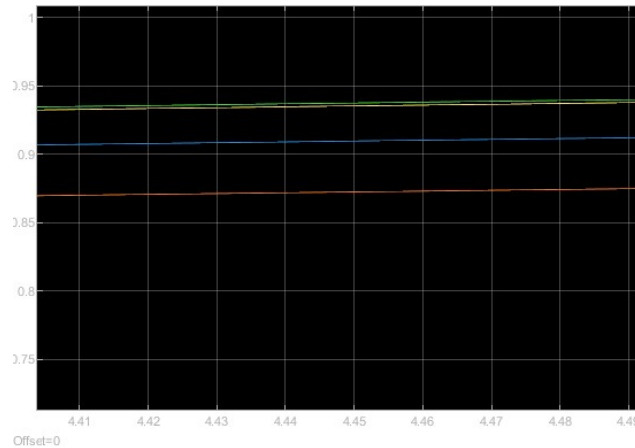


Figure 13: Zoom on the step function

Interestingly, while there is a noticeable time delay, the trajectories remain fairly close to the true curves (as shown in zoom 13) and converge rapidly to the true states. In the case of the local observer version, the ramp signal for velocity is particularly noteworthy: we observe an inversion where the last agent, around 2.5 seconds, overshoots its target trajectory and then gradually converges to the true state.

A ring topology connects nodes in a loop. It's a bit more resilient than a chain, especially if data can travel in both directions. Still, adding or removing nodes isn't very flexible, and depending on where the node is in the chain it may be slower to update or track the reference.

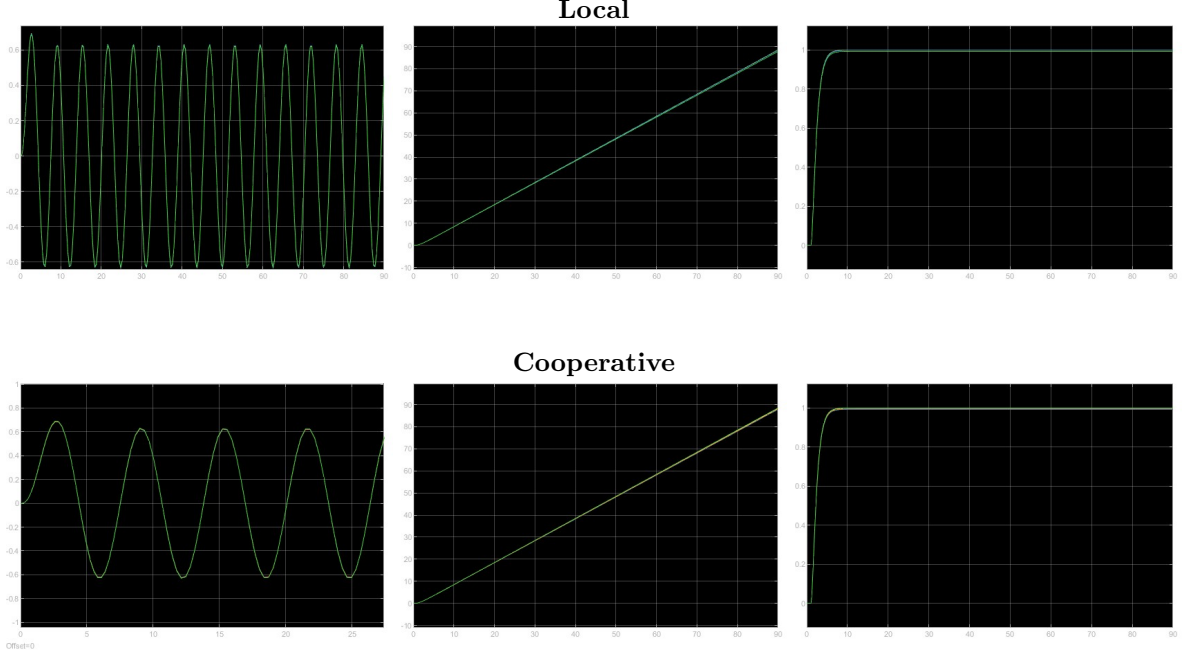


Figure 14: Positions in Ring Topology structure under local and cooperative estimation strategies.

Mesh topology connects nodes more freely. Some or all nodes may have multiple links, which makes it highly reliable. It scales well and keeps latency low, but it's harder to build and manage.

A fully connected topology is the extreme case of a mesh: every node connects directly to every other node. It's the most reliable and fastest setup, but it doesn't scale—it quickly becomes impractical as the network grows.

Star topology connects all nodes to a central hub, which is then connected to the leader node. It's easy to expand and simple to manage. The downside is that everything depends on the central node: if it fails, the whole system can go down, because communication between nodes also has to pass through that central point. The implementation is not done as it is as fast as the fully connected topology but offering more scalability and reduced costs, costing the designer robustness, while receiving a payoff in terms of performances.

For this report (as anticipated in the previous sections), we chose to analyze the mesh structure. While it is not as fast as the fully connected or star topologies, this allows us to observe more significant differences in the outputs of individual nodes. Moreover, the mesh topology represents a more realistic scenario for communication networks, as it offers better scalability and can grow organically with the network's demands. From a cost perspective, full connectivity between nodes is not required; information can propagate through the network via a limited number of intermediary or "relay" nodes, making the structure more scalable and resource-efficient. In terms of effectiveness, regions with higher connectivity enable neighboring nodes to refine their state estimates by incorporating more accurate and redundant information, thus improving overall estimation accuracy across the network.

5 Effect of Design Parameters

In this section we aim to test different values for the design parameters in our algorithms.

5.1 Coupling gain c

This design parameter needs to be over a certain value for the algorithm to converge to the reference given by the leader node. It is particularly important as it needs to be over a certain quota for the error to converge to zero:

$$c \geq \frac{1}{2\min_{i \in N} \text{Re}(\lambda_i)}$$

In our case the value c must be greater or equal to is 0.5, so in the graphs it is clear that as we increase it, it converges much more rapidly than in previous iterations.

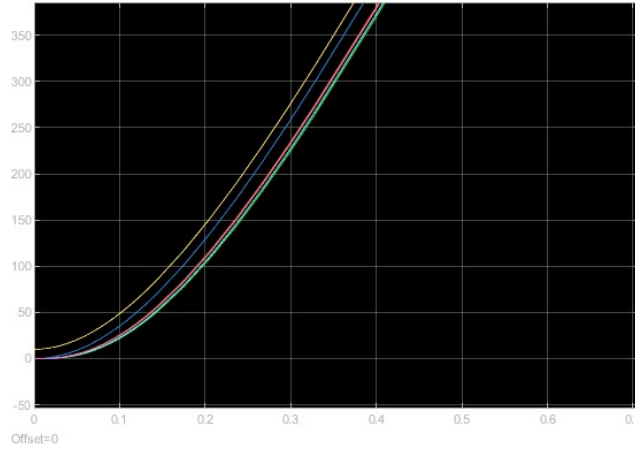


Figure 15: Velocity of a ramp, zoomed

For our case of study we chose to adapt the mesh topology even though the difference if we used chain would be more noticeable, because as we can see in image 15 the delays in time of each agent seem more adherent to the leader node (the trajectory on top). An interesting observation is also one regarding the grouping of such trajectories, as they seem to form a group that's pretty much overlapping with each other. If we apply these property in a scenario of guiding a group of cyber-physical systems this behavior is important, as even though the agents don't immediately follow the leader (as it would happen in a fully connected topology) they at least maintain a behavior that's consistent with each other even using the minimum value of the design parameter c . As will be shown in the next images, an increment of c corresponds to an increment in precision when it comes to tracking at the cost of increased energy consumption.

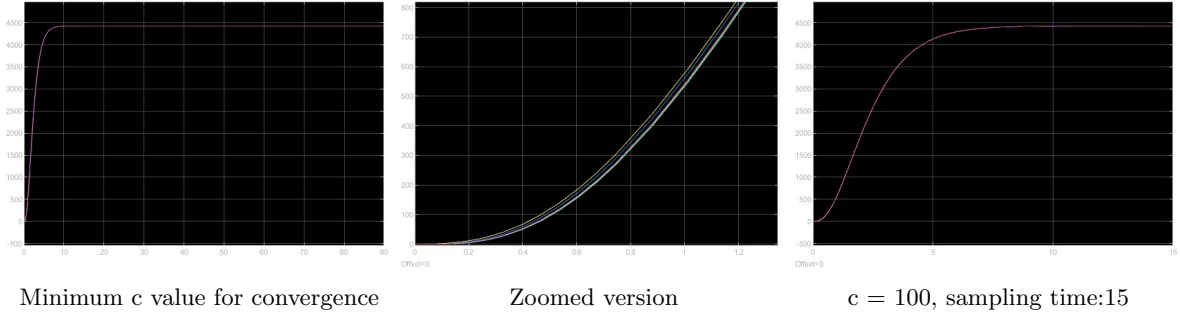


Figure 16: Cooperative Observers

5.2 Weighting Matrix Q & R

The design parameters Q and R represent respectively the energy of the state and the energy of the command, they both take an important role in the design of the distributed controller as they are needed to solve the algebraic Riccati equation giving us the gain K for both the local and the cooperative observers. They are both symmetric matrices (typically diagonal) and always positive, they play an important role in the computation of $u(t)$:

$$u(t) = \arg \min_{u(t)} \left[\frac{1}{2} \cdot \int_0^{+\infty} (x^T(t)Qx(t) + u^T(t)Ru(t)) dt \right]$$

subject to

$$\dot{x}(t) = Ax(t) + Bu(t)$$

In this context, Q and R act as weighting coefficients: increasing one or the other influences the system's behavior by reshaping the cost function landscape. However, it is important to highlight that what truly matters is not their absolute magnitude, but the relative weighting, that is, the ratio Q/R . During this analysis we have just modified R in order to influence our ratio, keeping Q equal to the identity ($Q = I$).

Cooperative

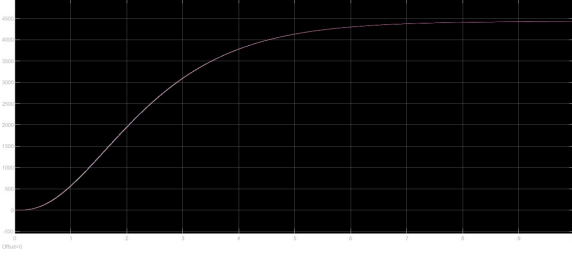


Figure 17: Positions $R = 0.01$

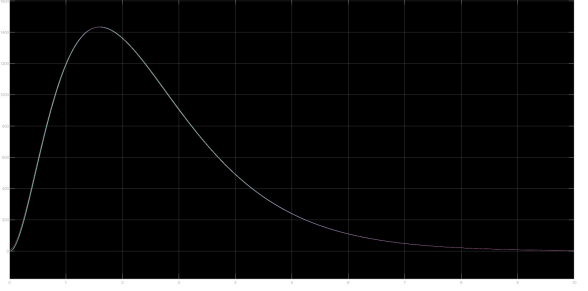


Figure 18: Velocities - $R = 0.01$

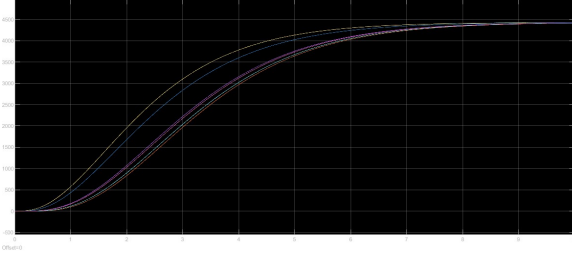


Figure 19: Positions - $R = 100$

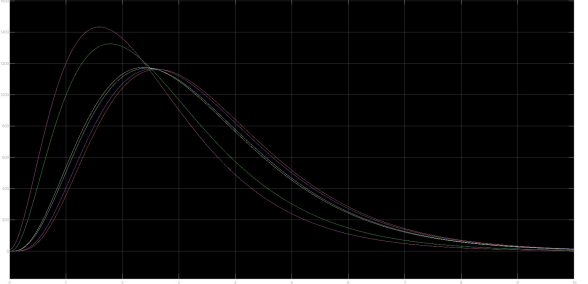


Figure 20: Velocities - $R = 100$

The tuning of the weighting matrices Q and R , specifically their ratio Q/R , as we said, shapes the controller's behavior. A high Q/R ratio leads to aggressive, fast responses with potential overshoot and higher actuator usage, ideal when quick convergence is essential and energy constraints are relaxed. Conversely, a low Q/R ratio results in a more cautious, slower, and heavily damped response that minimizes overshoot and reduces energy consumption, which is preferable in energy-limited applications or where hardware longevity is important. Therefore, selecting the appropriate balance between Q and R is crucial to achieving the desired trade-off between performance and resource utilization in control system design.

As part of this analyzes we report also the value assume by the input defined by our control law of a single agent.

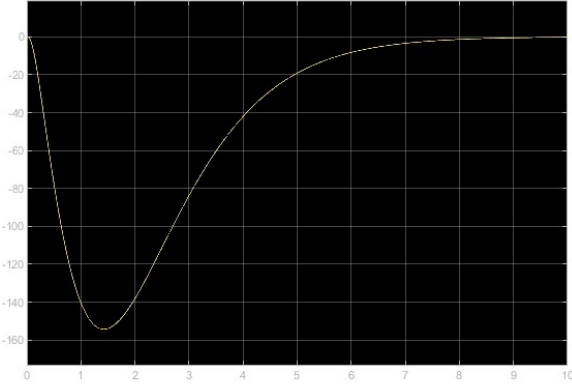


Figure 21: Input - $R = 100$

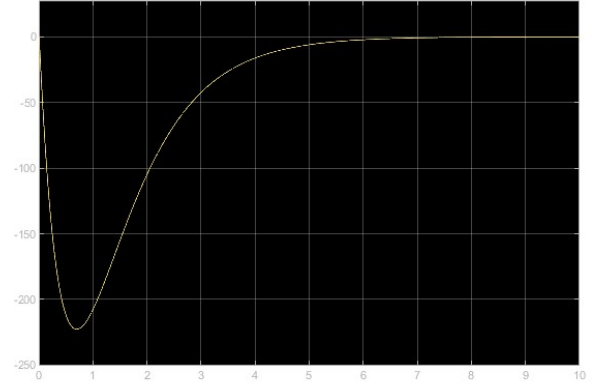


Figure 22: Input - $R = 0.01$

Local

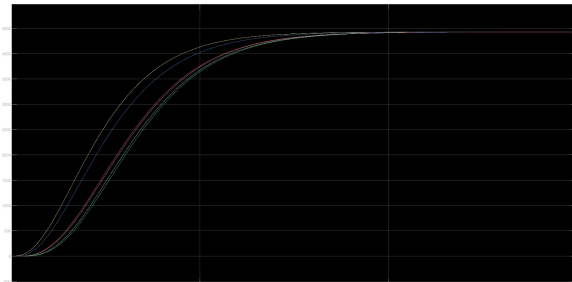


Figure 23: Positions - $R = 100$

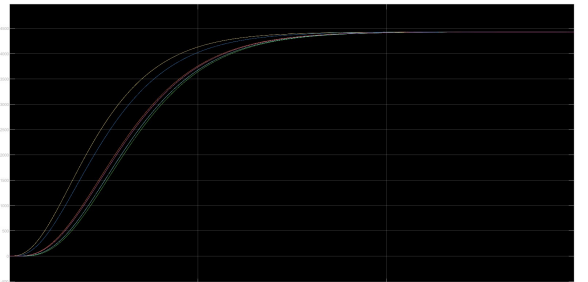


Figure 24: Positions - $R = 100$

We also observe that, by comparing the two algorithms, using a small R in the case of the local observer results in a slower transient response compared to the cooperative observer.

6 Effect of Statistical Noise η

In this section, we provide a brief analysis of the impact of noise on the performance of the program, examining specific cases that are considered particularly relevant.

Case in point we want to see the most basic case of noise on the leader node, we expect it to generate a distorted reference that will be followed by the other agents.

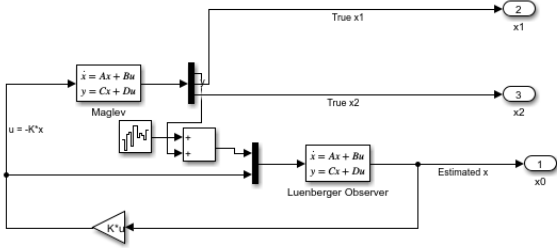


Figure 25: White noise added on the leader node

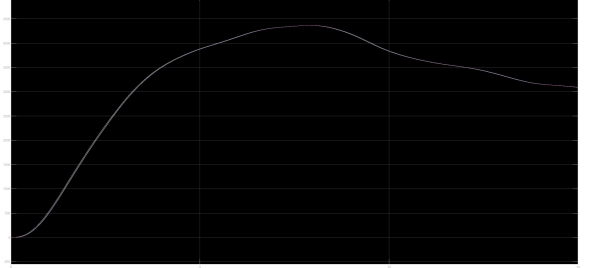


Figure 26: Position trajectory for cooperative neighborhood observer (mesh topology)

As observed, the trajectory is significantly distorted, making it difficult to recognize that it should follow a step reference. Despite operating within a cooperative structure, the algorithm exhibits no self-correction. This behavior occurs because the system assumes it is performing the intended task correctly.

We also introduced noise at the S1 node of a star topology. Our hypothesis was that this should change the trajectory of the other agents, given that the first agent should give the other agents a noisy output.

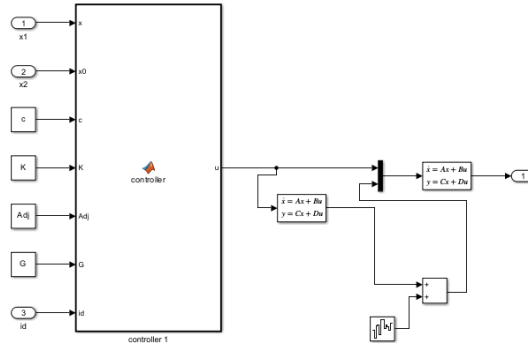


Figure 27: White noise added on the agents

For testing purposes we changed the topology of the agents to be a chain, we noticed that even though noise was only put on the first node the sixth was also getting wrong information. This is to be expected as all the information travels in a pipe of follower agents.

This is done with local observers as with the cooperative observers the results shouldn't vary much, given that node 6 can't give informations to node 5 to correct trajectories.

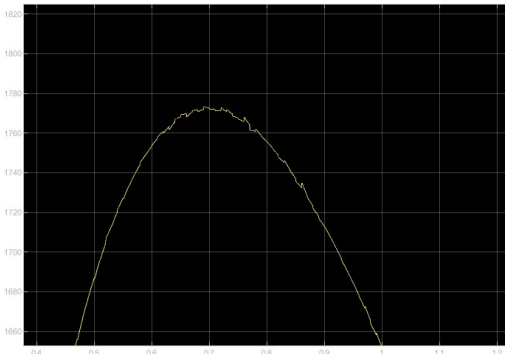


Figure 28: Zoom on the top part of the agent 6, with noise in the node 1

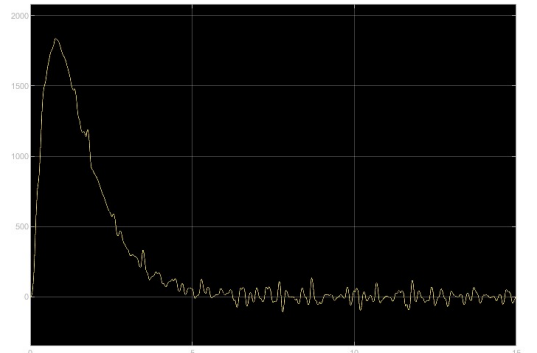


Figure 29: Error on the output of node 6